Tối ưu siêu tham số SVM bằng thuật toán Cuckoo Search cho bài toán phân loại văn bản tiếng Việt

Quốc Bảo Vũ[1], Trung Khánh Nguyễn[1], Nguyễn Minh Tuấn[1*]

[1*]Khoa Công nghệ thông tin, Học Viện Công Nghệ Bưu Chính Viễn Thông, Thành phố Hồ Chí Minh, 70000,Việt Nam.

*Corresponding author(s). E-mail(s): minhtuan@ptit.edu.vn;
Contributing authors: baovq2003@gmail.com; nguyentrungk461@gmail.com;

Tóm tắt

Phân loại văn bản tiếng Việt là một nhiệm vụ quan trọng trong xử lý ngôn ngữ tự nhiên với nhiều ứng dụng như gợi ý tin tức và phân loại chủ đề. Support Vector Machine (SVM) là một mô hình hiệu quả cho bài toán này; tuy nhiên, hiệu suất của SVM phụ thuộc mạnh vào việc lựa chọn các siêu tham số như tham số phạt C, hệ số kernel γ và loại kernel. Việc điều chỉnh thủ công hoặc sử dụng giá trị mặc định thường không mang lại kết quả tối ưu. Nghiên cứu này đề xuất ứng dụng thuật toán Cuckoo Search (CS), một thuật toán tối ưu meta-heuristic dựa trên hành vi sinh sản ký sinh của chim cu cu, nhằm tối ưu hóa các siêu tham số của mô hình SVM cho bài toán phân loại văn bản tiếng Việt cụ thể là phân loại chủ đề cho tin tức tiếng Việt. Thực nghiệm được tiến hành trên tập dữ liệu Vietnamese Online News Dataset gồm 37.860 văn bản thuộc 10 chủ đề khác nhau. Các siêu tham số của SVM được tối ưu trong không gian tìm kiếm toàn cục bằng Cuckoo Search. Kết quả cho thấy mô hình CS-SVM đạt độ chính xác 87% và F1-score 0.87, cao hơn so với mô hình SVM sử dụng tham số mặc định (84% và F1-score 0.84). Mức cải thiện 3% về độ chính xác cho thấy hiệu quả của thuật toán Cuckoo Search trong việc tối ưu siêu tham số SVM cho bài toán phân loại văn bản tiếng Việt.

Từ khóa: Cuckoo Search, Support Vector Machines (SVM), Tối ưu siêu tham số, Meta-heuristic, Phân loại văn bản tiếng Việt, Phân loại chủ đề tin tức, Tiếng Việt

1 Giới thiệu

Tối ưu siêu tham số là bước quan trọng trong quy trình học máy, ảnh hưởng trực tiếp đến hiệu suất và khả năng tổng quát hóa của mô hình [1]. Khác với các tham số được học trong quá trình huấn luyện, siêu tham số được thiết lập trước và hướng dẫn chính quá trình huấn luyện. Việc lựa chọn bộ siêu tham số tối ưu đảm bảo mô hình không chỉ khớp tốt với dữ liệu huấn luyện mà còn tạo ra kết quả tốt trên dữ liệu mới.

Các phương pháp truyền thống như Grid Search và Random Search gặp nhiều hạn chế. Grid Search tìm kiếm toàn diện tất cả các tổ hợp siêu tham số nhưng yêu cầu thời gian rất dài và chịu ảnh hưởng của lời nguyền chiều cao khi số lượng siêu tham số tăng lên [2]. Random Search tuy tránh được bản chất toàn diện nhưng vẫn gặp khó khăn trong không gian nhiều chiều do xác suất lấy mẫu gần vùng tối ưu giảm xuống [3]. Cả hai phương pháp đều thiếu cơ chế thông minh để khai thác các vùng hứa hẹn trong không gian tìm kiếm. Các thuật toán meta-heuristic lấy cảm hứng từ tự nhiên như Genetic Algorithm [4], Particle Swarm Optimization [5], và Ant Colony Optimization [6] đã được đề xuất để cân bằng giữa khám phá và khai thác không gian tìm kiếm, cải thiện đáng kể hiệu quả tối ưu siêu tham số [7].

Support Vector Machine (SVM) là thuật toán học máy có giám sát mạnh mẽ, tìm siêu phẳng tốt nhất để phân tách các lớp bằng cách tối đa hóa lề [8]. SVM hiệu quả cho các nhiệm vụ phân loại phức tạp nhờ khả năng xử lý dữ liệu có số chiều cao [9]. Tuy nhiên, hiệu suất của SVM rất nhạy cảm với việc lựa chọn siêu tham số, đặc biệt là tham số điều chuẩn C và hệ số kernel γ của kernel RBF [10]. Tham số C kiểm soát sự đánh đổi giữa lỗi huấn luyện và khả năng tổng quát hóa, trong khi *gamma* xác định ảnh hưởng của các điểm huấn luyện riêng lẻ. Việc lựa chọn tổ hợp tối ưu của C và γ là nhiệm vụ phức tạp, đặc biệt khi không gian tìm kiếm lớn.

Trong số các thuật toán meta-heuristic, Cuckoo Search (CS) được giới thiệu bởi Yang và Deb [11] đã thu hút sự chú ý nhờ cơ chế tìm kiếm hiệu quả dựa trên hành vi sinh sản ký sinh của chim cu cu và chuyển động Lévy flights. CS có ưu điểm sử dụng ít tham số điều chỉnh hơn so với PSO và GA, đồng thời có khả năng thoát khỏi các điểm tối ưu cục bộ tốt hơn nhờ Lévy flights - dạng bước nhảy ngẫu nhiên cho phép thuật toán thực hiện cả bước nhảy ngắn (khai thác cục bộ) và bước nhảy dài (khám phá toàn cục) [12, 13]. Yang và Deb [14] chứng minh CS có tốc độ hội tụ nhanh hơn nhiều thuật toán meta-heuristic khác, trong khi Mareli và Twala [15] phát triển phiên bản cải tiến với cơ chế thích ứng. Việc kết hợp CS với SVM đã được áp dụng thành công trong nhiều lĩnh vực: nhận dạng lỗi thiết bị tàu biển [16], chẩn đoán ung thư phổi [17], chẩn đoán bệnh lâm sàng [18], và gần đây nhất là phân loại ảnh vệ tinh [7]. Các nghiên cứu này chứng minh CS có thể tìm được bộ siêu tham số tối ưu cho SVM hiệu quả hơn Grid Search và Random Search.

Mặc dù đã có nhiều nghiên cứu về tối ưu siêu tham số SVM bằng thuật toán meta-heuristic, hầu hết tập trung vào các ngôn ngữ phổ biến hoặc các bài toán trong y tế và xử lý ảnh. Nghiên cứu về việc áp dụng CS để tối ưu SVM cho phân loại văn bản tiếng Việt vẫn còn hạn chế. Tiếng Việt có những đặc thù riêng về ngữ pháp (ngôn ngữ đơn lập, không biến hình từ), cấu trúc từ (nhiều từ ghép, từ láy), hệ thống thanh điệu phức tạp và ngữ nghĩa phụ thuộc nhiều vào ngữ cảnh. Văn bản tin tức tiếng Việt có đặc điểm riêng với sự đa dạng về chủ đề, phong cách viết báo chí và sử dụng nhiều thuật ngữ chuyên ngành. Hơn nữa, các nghiên cứu trước đây thường so sánh CS-SVM với một hoặc hai phương pháp baseline đơn giản, chưa có đánh giá chi tiết về hiệu suất trên dữ liệu văn bản tiếng Việt với nhiều chủ đề đa dạng.

Để lấp đầy khoảng trống này, bài báo đề xuất ứng dụng Cuckoo Search để tối ưu hóa siêu tham số của SVM cho phân loại văn bản tiếng Việt. Nghiên cứu được thực hiện trên Vietnamese Online News Dataset với 37.860 văn bản thuộc 10 chủ đề đa dạng. Các đóng góp chính bao gồm: (1) Đề xuất phương pháp CS-SVM cho phân loại văn bản tiếng Việt, trong đó CS tự động tìm kiếm bộ siêu tham số tối ưu bao gồm C, *gamma* và loại kernel; (2) Thực hiện thực nghiệm toàn diện với biểu diễn văn bản dựa trên PhoBERT [19], đánh giá hiệu suất thông qua các chỉ số accuracy, precision, recall và F1-score; (3) So sánh chi tiết hiệu quả của

CS-SVM với SVM sử dụng tham số mặc định, phân tích cải thiện về độ chính xác tổng thể và hiệu suất trên từng chủ đề; (4) Đánh giá chi phí tính toán và thời gian thực thi, đảm bảo tính khả thi cho các ứng dụng thực tế.

2 Phương pháp nghiên cứu

2.1 Support Vector Machine (SVM)

Support Vector Machine là một thuật toán học máy có giám sát mạnh mẽ, được áp dụng rộng rãi trong các bài toán phân loại nhờ khả năng xử lý hiệu quả dữ liệu có số chiều cao [8]. Ý tưởng cốt lõi của SVM là xác định siêu phẳng tối ưu trong không gian đặc trưng để phân tách các điểm dữ liệu thuộc các lớp khác nhau, đồng thời tối đa hóa khoảng cách (lề) từ siêu phẳng đến các điểm dữ liệu gần nhất của mỗi lớp [9].

2.1.1 Bài toán phân loại nhị phân

Cho tập dữ liệu huấn luyện $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, trong đó $\mathbf{x}_i \in \mathbb{R}^d$ là vector đặc trưng của mẫu thứ i, $y_i \in \{-1, +1\}$ là nhãn lớp tương ứng, và N là số lượng mẫu trong tập huấn luyện. Mục tiêu của SVM là tìm một siêu phẳng phân tách sao cho khoảng cách từ siêu phẳng đến các điểm dữ liệu gần nhất (support vectors) được tối đa hóa.

Siêu phẳng phân tách được biểu diễn bởi phương trình:

$$f(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b = 0, \tag{1}$$

trong đó $\mathbf{w} \in \mathbb{R}^d$ là vector trọng số vuông góc với siêu phẳng, và $b \in \mathbb{R}$ là hệ số điều chỉnh (bias).

Để tìm siêu phẳng tối ưu, SVM giải bài toán tối ưu sau:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2, \tag{2}$$

với ràng buộc:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1, \quad \forall i = 1, \ldots, N. \tag{3}$$

Hàm quyết định phân loại được định nghĩa như sau:

$$\hat{y} = \text{sign}(\mathbf{w}^T\mathbf{x} + b). \tag{4}$$

Bài toán tối ưu trên áp dụng cho trường hợp dữ liệu phân tách tuyến tính hoàn toàn (hard margin SVM), không cho phép bất kỳ điểm dữ liệu nào bị phân loại sai.

2.1.2 Soft Margin SVM

Trong thực tế, dữ liệu thường không phân tách tuyến tính hoàn toàn do nhiễu hoặc sự chồng chéo giữa các lớp. Để xử lý trường hợp này, SVM sử dụng khái niệm lề mềm (soft margin) bằng cách đưa vào các biến slack $\xi_i \geq 0$ cho phép một số điểm dữ liệu vi phạm ràng buộc lề.

Bài toán tối ưu được điều chỉnh thành:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i, \tag{5}$$

với ràng buộc:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i = 1, \ldots, N. \tag{6}$$

Trong công thức trên, $C > 0$ là siêu tham số điều chuẩn (regularization parameter), đóng vai trò cân bằng giữa hai mục tiêu: (1) tối đa hóa lề phân loại (thông qua việc tối thiểu hóa $\|\mathbf{w}\|^2$), và (2) tối thiểu hóa tổng lỗi phân loại trên tập huấn luyện (thông qua việc tối thiểu hóa $\sum_{i=1}^{N}\xi_i$) [10]. Giá trị C lớn sẽ làm cho mô hình tập trung vào việc phân loại chính xác các điểm huấn luyện, có thể dẫn đến overfitting. Ngược lại, giá trị C nhỏ cho phép mô hình chấp nhận nhiều lỗi phân loại hơn trên tập huấn luyện, tạo ra mô hình đơn giản hơn nhưng có thể dẫn đến underfitting.

2.1.3 Dạng đối ngẫu và Support Vectors

Bài toán tối ưu SVM có thể được biểu diễn dưới dạng đối ngẫu (dual form) thông qua việc sử dụng các nhân tử Lagrange $\alpha_i \geq 0$:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{N}\alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j, \tag{7}$$

với ràng buộc:

$$\sum_{i=1}^{N}\alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad \forall i = 1, \ldots, N. \tag{8}$$

Các điểm dữ liệu có $\alpha_i > 0$ được gọi là support vectors và đóng vai trò quyết định trong việc xác định siêu phẳng phân tách. Đây là một đặc điểm quan trọng của SVM: chỉ một tập hợp nhỏ các điểm dữ liệu (support vectors) ảnh hưởng đến mô hình cuối cùng, giúp SVM có khả năng tổng quát hóa tốt.

2.1.4 Kernel và ánh xạ phi tuyến

Đối với các bài toán mà dữ liệu không phân tách tuyến tính trong không gian đặc trưng gốc, SVM sử dụng kỹ thuật kernel để ánh xạ dữ liệu vào không gian có số chiều cao hơn, nơi dữ liệu có thể được phân tách tuyến tính. Thay vì tính toán trực tiếp ánh xạ $\phi(\mathbf{x})$, SVM sử dụng hàm kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T\phi(\mathbf{x}_j)$ để tính tích vô hướng trong không gian mới mà không cần biểu diễn tường minh ánh xạ ϕ.

Một trong những hàm kernel hiệu quả và được sử dụng phổ biến nhất là Radial Basis Function (RBF) kernel, còn được gọi là Gaussian kernel:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2\right), \tag{9}$$

trong đó $\|\mathbf{x}_i - \mathbf{x}_j\|$ là khoảng cách Euclid giữa hai điểm dữ liệu, và $\gamma > 0$ là siêu tham số kiểm soát độ rộng của kernel.

Khi sử dụng kernel, bài toán đối ngẫu trở thành:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j), \tag{10}$$

với ràng buộc tương tự như phương trình (8).

Siêu tham số γ có ảnh hưởng quan trọng đến hiệu suất của SVM với RBF kernel [10]. Giá trị γ lớn tạo ra kernel hẹp, dẫn đến ranh giới quyết định phức tạp và chi tiết, có thể gây ra overfitting do mô hình quá nhạy cảm với các điểm dữ liệu riêng lẻ. Ngược lại, giá trị γ nhỏ tạo ra kernel rộng, dẫn đến ranh giới quyết định mượt mà và tổng quát hơn, nhưng có thể gây ra underfitting nếu quá đơn giản.

2.1.5 Phân loại đa lớp

Trong nghiên cứu này, bài toán phân loại chủ đề tin tức tiếng Việt là bài toán phân loại đa lớp với 10 lớp khác nhau. SVM ban đầu được thiết kế cho bài toán phân loại nhị phân, do đó cần có chiến lược để mở rộng sang bài toán đa lớp. Có hai chiến lược phổ biến: One-vs-One (OvO) và One-vs-Rest (OvR).

Nghiên cứu này sử dụng chiến lược One-vs-One (OvO). Với K lớp, chiến lược OvO xây dựng $\frac{K(K-1)}{2}$ bộ phân loại nhị phân, mỗi bộ phân loại được huấn luyện để phân biệt giữa một cặp lớp. Trong trường hợp của nghiên cứu này với 10 lớp, tổng cộng có $\frac{10 \times 9}{2} = 45$ bộ phân loại nhị phân được xây dựng. Khi phân loại một mẫu mới, mỗi bộ phân loại nhị phân đưa ra dự đoán, và lớp cuối cùng được quyết định theo nguyên tắc bỏ phiếu (voting): lớp nhận được nhiều phiếu nhất từ các bộ phân loại sẽ được chọn làm kết quả cuối cùng.

2.1.6 Tầm quan trọng của việc tối ưu siêu tham số

Từ các phân tích trên, có thể thấy rằng hiệu suất của SVM phụ thuộc rất nhiều vào việc lựa chọn các siêu tham số, đặc biệt là C và γ khi sử dụng RBF kernel [1, 10]. Việc lựa chọn không phù hợp các siêu tham số này có thể dẫn đến:

- **Overfitting:** Khi C quá lớn hoặc γ quá lớn, mô hình sẽ khớp quá mức với dữ liệu huấn luyện, dẫn đến hiệu suất kém trên dữ liệu mới.
- **Underfitting:** Khi C quá nhỏ hoặc γ quá nhỏ, mô hình sẽ quá đơn giản và không nắm bắt được đầy đủ cấu trúc của dữ liệu.

Do không có cách nào để biết trước bộ siêu tham số (C, γ) tối ưu cho một bài toán cụ thể [10], việc tìm kiếm tự động bộ siêu tham số này thông qua các thuật toán tối ưu là cần thiết. Đây chính là động lực cho việc áp dụng thuật toán Cuckoo Search trong nghiên cứu này để tối ưu các siêu tham số của SVM, nhằm cải thiện hiệu suất phân loại văn bản tiếng Việt.

2.2 PhoBERT

Để áp dụng SVM cho bài toán phân loại văn bản, bước quan trọng đầu tiên là chuyển đổi văn bản từ dạng chuỗi ký tự thành vector đặc trưng số học. Trong nghiên cứu này, chúng tôi sử dụng PhoBERT [19] - một mô hình ngôn ngữ được huấn luyện trước (pre-trained language model) chuyên biệt cho tiếng Việt - để trích xuất các biểu diễn ngữ nghĩa sâu của văn bản.

2.2.1 Mô hình PhoBERT

PhoBERT là mô hình ngôn ngữ dựa trên kiến trúc BERT (Bidirectional Encoder Representations from Transformers) được huấn luyện trước trên kho ngữ liệu tiếng Việt lớn [19]. Khác với các phương pháp biểu diễn văn bản truyền thống như Bag-of-Words hay TF-IDF chỉ nắm bắt thông tin thống kê về tần suất xuất hiện của từ, PhoBERT có khả năng học các biểu diễn ngữ cảnh sâu (deep contextualized representations) thông qua cơ chế self-attention đa đầu (multi-head self-attention) và kiến trúc Transformer.

PhoBERT được huấn luyện trước bằng hai nhiệm vụ chính:

- **Masked Language Modeling (MLM):** Dự đoán các từ bị che (mask) trong câu dựa trên ngữ cảnh xung quanh, giúp mô hình học được quan hệ ngữ nghĩa giữa các từ.
- **Next Sentence Prediction (NSP):** Dự đoán xem hai câu có liên tiếp nhau trong văn bản gốc hay không, giúp mô hình hiểu được mối quan hệ giữa các câu.

Nguyen và Nguyen [19] đã huấn luyện PhoBERT trên tập dữ liệu gồm 20GB văn bản tiếng Việt từ Wikipedia và các nguồn tin tức trực tuyến. Mô hình đã chứng minh hiệu suất vượt trội so với các mô hình đa ngôn ngữ như mBERT (multilingual BERT) trên nhiều tác vụ xử lý ngôn ngữ tự nhiên tiếng Việt, bao gồm gán nhãn từ loại (POS tagging), nhận dạng thực thể có tên (NER), và phân loại văn bản.

2.2.2 Quy trình trích xuất đặc trưng

Trong nghiên cứu này, chúng tôi sử dụng PhoBERT-base với cấu hình 12 lớp Transformer, 768 đơn vị ẩn (hidden units), và 12 attention heads. Quy trình trích xuất đặc trưng từ văn bản gồm các bước sau:

Bước 1: Tiền xử lý văn bản

Văn bản đầu vào được chuẩn hóa bằng cách:

- Loại bỏ các ký tự đặc biệt không cần thiết
- Chuẩn hóa khoảng trắng và dấu câu
- Chuyển về chữ thường.
- Chuẩn hóa unicode NFC
- Chuẩn hóa văn bản
- Tách từ thành các từ đơn hoặc từ ghép

Bước 2: Tokenization

Văn bản được phân tách thành các token (từ hoặc subword) sử dụng bộ tokenizer của PhoBERT dựa trên thuật toán BPE (Byte Pair Encoding). Đối với mỗi văn bản, chuỗi token được thêm các token đặc biệt:

- `[CLS]`: Token đặc biệt được thêm vào đầu chuỗi, biểu diễn của nó được sử dụng cho các tác vụ phân loại câu
- `[SEP]`: Token phân tách được thêm vào cuối chuỗi

Bước 3: Encoding với PhoBERT

Chuỗi token được đưa qua các lớp Transformer của PhoBERT. Đối với mỗi token, mô hình tạo ra một vector biểu diễn 768 chiều. Đặc biệt, biểu diễn của token `[CLS]` ở lớp cuối cùng được thiết kế để nắm bắt thông tin ngữ nghĩa của toàn bộ câu/đoạn văn.

Gọi $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{768}$ là vector biểu diễn của token `[CLS]` tại lớp đầu ra cuối cùng của PhoBERT. Vector này chứa thông tin ngữ nghĩa tổng hợp của toàn bộ văn bản đầu vào và được sử dụng làm vector đặc trưng cho văn bản trong mô hình SVM.

Bước 4: Vector hóa cho SVM

Với mỗi văn bản d_i trong tập dữ liệu, chúng tôi thu được vector đặc trưng:

$$\mathbf{x}_i = \text{PhoBERT}(d_i) = \mathbf{h}_{[\text{CLS}]}^{(i)} \in \mathbb{R}^{768} \tag{11}$$

Vector $\mathbf{x}_i$ này sau đó được sử dụng làm đầu vào cho mô hình SVM để huấn luyện và dự đoán.

2.2.3 Ưu điểm của PhoBERT cho tiếng Việt

Việc sử dụng PhoBERT mang lại nhiều lợi thế cho bài toán phân loại văn bản tiếng Việt:

- **Nắm bắt ngữ nghĩa sâu:** PhoBERT học được các biểu diễn dựa trên ngữ cảnh, giúp phân biệt các từ đồng âm khác nghĩa phổ biến trong tiếng Việt.
- **Xử lý từ ghép:** Tiếng Việt có nhiều từ ghép và cụm từ cố định. Cơ chế subword tokenization của PhoBERT có thể xử lý hiệu quả các đơn vị ngôn ngữ này.
- **Biểu diễn cấp câu:** Token `[CLS]` tổng hợp thông tin từ toàn bộ câu thông qua cơ chế attention, nắm bắt được cấu trúc và ngữ nghĩa của câu tiếng Việt.
- **Transfer learning:** PhoBERT được huấn luyện trước trên lượng lớn dữ liệu tiếng Việt, cho phép chuyển giao kiến thức ngôn ngữ tổng quát sang tác vụ phân loại chủ đề tin tức cụ thể.
- **Xử lý từ vựng mở:** Khác với các phương pháp dựa trên từ điển cố định, PhoBERT có thể xử lý các từ mới hoặc từ hiếm thông qua tokenization ở mức subword.

2.2.4 Tích hợp PhoBERT và SVM

Mô hình tổng thể của nghiên cứu có thể được mô tả như sau: PhoBERT đóng vai trò là bộ trích xuất đặc trưng (feature extractor), chuyển đổi văn bản thô thành các vector đặc trưng có số chiều cố định, trong khi SVM đóng vai trò là bộ phân loại (classifier) học ánh xạ từ không gian đặc trưng sang các nhãn lớp.

Trong quá trình huấn luyện, các tham số của PhoBERT được giữ cố định (frozen), và chỉ có các tham số của SVM (bao gồm các support vectors và siêu tham số C, γ) được tối ưu. Cách tiếp cận này có hai lợi ích:

- **Giảm chi phí tính toán:** Không cần fine-tune PhoBERT, giúp tiết kiệm đáng kể thời gian và tài nguyên tính toán
- **Tránh overfitting:** Giữ nguyên các trọng số đã được huấn luyện tốt của PhoBERT, tránh làm giảm khả năng tổng quát hóa khi tập dữ liệu huấn luyện nhỏ

Việc tối ưu siêu tham số của SVM thông qua thuật toán Cuckoo Search sẽ giúp tìm ra bộ tham số (C, γ) tối ưu để khai thác hiệu quả nhất các đặc trưng trích xuất từ PhoBERT, từ đó nâng cao hiệu suất phân loại chủ đề tin tức tiếng Việt.

2.3 Thuật toán tối ưu Cuckoo Search (CSO)

Cuckoo Search (CS) là một thuật toán tối ưu meta-heuristic được Yang và Deb đề xuất vào năm 2009 [11], lấy cảm hứng từ hành vi sinh sản ký sinh của chim cu cu kết hợp với chuyển động Lévy flights quan sát được trong tự nhiên. Thuật toán này đã chứng minh hiệu quả trong việc giải quyết các bài toán tối ưu phức tạp và được áp dụng thành công trong nhiều lĩnh vực [14].

2.3.1 Cơ sở sinh học

Hành vi sinh sản của chim cu cu có những đặc điểm độc đáo làm cơ sở cho thuật toán CS:

- **Sinh sản ký sinh:** Chim cu cu đặt trứng vào tổ của các loài chim khác (chim chủ). Trứng cu cu thường có màu sắc và kích thước tương tự với trứng của chim chủ để tránh bị phát hiện.
- **Cơ chế phát hiện:** Chim chủ có thể phát hiện trứng lạ trong tổ với một xác suất nhất định. Nếu trứng cu cu bị phát hiện, chim chủ sẽ vứt bỏ trứng đó hoặc bỏ tổ và xây tổ mới ở nơi khác.
- **Chiến lược di chuyển:** Chim cu cu sử dụng chuyển động Lévy flights - một dạng bước đi ngẫu nhiên với sự kết hợp giữa các bước nhảy ngắn thường xuyên và các bước nhảy dài không thường xuyên - để tìm kiếm tổ phù hợp.

2.3.2 Lévy Flights

Lévy flights là một dạng bước đi ngẫu nhiên trong đó độ dài bước nhảy được lấy mẫu từ phân phối Lévy - một phân phối xác suất có đuôi dài (heavy-tailed distribution). Trong tự nhiên, nhiều loài động vật sử dụng mô hình chuyển động này khi tìm kiếm thức ăn, bao gồm chim biển, ong mật, và ngay cả con người trong một số hoạt động.

Phân phối Lévy được định nghĩa như sau:

$$L(s, \lambda) \sim s^{-\lambda}, \quad 1 < \lambda \leq 3, \tag{12}$$

trong đó s là độ dài bước nhảy và λ là tham số kiểm soát phân phối.

Trong thực tế, bước nhảy Lévy có thể được sinh ra bằng thuật toán Mantegna [13]:

$$s = \frac{u}{|v|^{1/\beta}}, \tag{13}$$

với $u \sim \mathcal{N}(0, \sigma_u^2)$, $v \sim \mathcal{N}(0, \sigma_v^2)$, và:

$$\sigma_u = \left[\frac{\Gamma(1+\beta)\sin(\pi\beta/2)}{\Gamma((1+\beta)/2)\beta 2^{(\beta-1)/2}} \right]^{1/\beta}, \quad \sigma_v = 1, \tag{14}$$

trong đó $\beta \in [1, 2]$ là tham số kiểm soát phân phối Lévy (thường chọn $\beta = 1.5$), và $\Gamma(\cdot)$ là hàm Gamma.

Lévy flights mang lại lợi thế quan trọng cho thuật toán tối ưu: các bước nhảy ngắn cho phép khai thác kỹ lưỡng vùng lân cận (exploitation), trong khi các bước nhảy dài giúp khám phá các vùng mới trong không gian tìm kiếm (exploration), từ đó tăng khả năng thoát khỏi các điểm tối ưu cục bộ [12].

2.3.3 Mô hình hóa thuật toán

Thuật toán Cuckoo Search được xây dựng dựa trên ba quy tắc lý tưởng:

1. Mỗi con cu cu đặt một quả trứng vào một tổ được chọn ngẫu nhiên.
2. Các tổ tốt nhất chứa trứng có chất lượng cao sẽ được giữ lại cho thế hệ tiếp theo.
3. Số lượng tổ khả dụng là cố định, và xác suất trứng của cu cu bị chim chủ phát hiện là $P_a \in [0, 1]$. Khi bị phát hiện, chim chủ có thể vứt trứng hoặc bỏ tổ để xây tổ mới.

Trong mô hình toán học, mỗi tổ tương ứng với một giải pháp ứng viên (candidate solution) trong không gian tìm kiếm, và trứng trong tổ đại diện cho một giải pháp. Chất lượng của giải pháp được đánh giá thông qua hàm mục tiêu (fitness function).

2.3.4 Thuật toán Cuckoo Search

Thuật toán Cuckoo Search thực hiện theo các bước sau:

Bước 1: Khởi tạo quần thể

Khởi tạo quần thể gồm n tổ (solutions) với vị trí ngẫu nhiên trong không gian tìm kiếm. Đối với bài toán tối ưu siêu tham số SVM, mỗi tổ $\mathbf{x}_i = (C_i, \gamma_i)$ đại diện cho một bộ siêu tham số, trong đó:

$$C_i \in [C_{\min}, C_{\max}], \quad \gamma_i \in [\gamma_{\min}, \gamma_{\max}]. \tag{15}$$

Vị trí ban đầu được khởi tạo ngẫu nhiên:

$$\mathbf{x}_i^{(0)} = \mathbf{x}_{\min} + \text{rand}(\mathbf{x}_{\max} - \mathbf{x}_{\min}), \tag{16}$$

trong đó rand là số ngẫu nhiên trong khoảng $[0, 1]$.

Bước 2: Đánh giá fitness

Đối với mỗi tổ $\mathbf{x}_i$, tính giá trị fitness $f(\mathbf{x}_i)$ bằng cách:

1. Huấn luyện mô hình SVM với siêu tham số (C_i, γ_i) trên tập huấn luyện
2. Đánh giá mô hình trên tập validation
3. Tính fitness: $f(\mathbf{x}_i) = -\text{accuracy}_{\text{validation}}$

Dấu âm được sử dụng vì thuật toán CS tối thiểu hóa hàm mục tiêu, trong khi chúng ta muốn tối đa hóa độ chính xác.

Lưu giữ tổ tốt nhất: $\mathbf{x}_{\text{best}} = \arg\min_i f(\mathbf{x}_i)$.

Bước 3: Sinh giải pháp mới bằng Lévy flights

Với mỗi tổ $\mathbf{x}_i$, sinh ra một giải pháp mới $\mathbf{x}_i^{\text{new}}$ bằng cách thực hiện Lévy flight xung quanh tổ tốt nhất hiện tại:

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \alpha \oplus \text{Lévy}(\lambda), \tag{17}$$

trong đó:

- $\alpha > 0$ là hệ số kích thước bước (step size), thường chọn $\alpha = 0.01$
- $\oplus$ là phép nhân từng phần tử (element-wise multiplication)
- Lévy(λ) là vector bước nhảy Lévy

Trong thực tế, bước cập nhật thường được thực hiện xung quanh tổ tốt nhất:

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \alpha \cdot s \odot (\mathbf{x}_{\text{best}}^{(t)} - \mathbf{x}_i^{(t)}), \tag{18}$$

trong đó s là bước nhảy Lévy được tính theo công thức (12), và $\odot$ là phép nhân từng phần tử.

Bước 4: Đánh giá và chọn lọc

Tính fitness của giải pháp mới $f(\mathbf{x}_i^{\text{new}})$. Nếu giải pháp mới tốt hơn (có fitness thấp hơn), thay thế giải pháp cũ:

$$\mathbf{x}_i^{(t+1)} = \begin{cases} \mathbf{x}_i^{\text{new}} & \text{nếu } f(\mathbf{x}_i^{\text{new}}) < f(\mathbf{x}_i^{(t)}), \\ \mathbf{x}_i^{(t)} & \text{ngược lại.} \end{cases} \tag{19}$$

Cập nhật tổ tốt nhất nếu tìm được giải pháp tốt hơn.

Bước 5: Loại bỏ giải pháp kém (Abandonment)

Một phần các giải pháp kém được thay thế bằng các giải pháp mới. Với xác suất P_a, mỗi tổ bị loại bỏ và được thay thế bằng một tổ mới:

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + r \odot (\mathbf{x}_j^{(t)} - \mathbf{x}_k^{(t)}), \tag{20}$$

trong đó $r \sim U(0,1)$ là số ngẫu nhiên, và j, k là hai chỉ số được chọn ngẫu nhiên khác nhau.

Cơ chế này được thực hiện bằng cách:

$$\mathbf{x}_i^{(t+1)} = \begin{cases} \mathbf{x}_i^{(t)} + r \odot (\mathbf{x}_j^{(t)} - \mathbf{x}_k^{(t)}) & \text{nếu rand} < P_a, \\ \mathbf{x}_i^{(t)} & \text{ngược lại.} \end{cases} \tag{21}$$

Bước 6: Kiểm tra điều kiện dừng

Thuật toán lặp lại các bước 3-5 cho đến khi đạt được một trong các điều kiện dừng:

- Đạt số vòng lặp tối đa $t_{\max}$
- Fitness của tổ tốt nhất không cải thiện sau một số vòng lặp liên tiếp
- Đạt mức fitness mong muốn

Giải pháp cuối cùng là bộ siêu tham số $(\hat{C}, \hat{\gamma})$ tương ứng với tổ tốt nhất $\mathbf{x}_{\text{best}}$.

2.3.5 Cấu hình tham số cho CS-SVM

Trong nghiên cứu này, thuật toán Cuckoo Search được cấu hình với các tham số cụ thể để tối ưu siêu tham số của SVM. Kích thước quần thể được thiết lập là $n = 20$ tổ (solutions), đủ lớn để đảm bảo tính đa dạng của không gian tìm kiếm nhưng vẫn giữ chi phí tính toán hợp lý. Số vòng lặp tối đa được đặt ở mức $t_{\max} = 5$ iterations để cân bằng giữa thời gian thực thi và khả năng hội tụ của thuật toán. Tỷ lệ phát hiện $P_a = 0.25$ xác định xác suất một tổ bị loại bỏ và thay thế bằng giải pháp mới, giúp duy trì sự cân bằng giữa khám phá (exploration) và khai thác (exploitation) trong quá trình tìm kiếm. Hệ số bước nhảy $\alpha = 0.01$ kiểm soát độ lớn của các bước di chuyển trong không gian tìm kiếm, trong khi tham số Lévy $\beta = 1.5$ xác định đặc tính của phân phối Lévy flights, cho phép thuật toán thực hiện cả các bước nhảy ngắn để khai thác cục bộ và các bước nhảy dài để khám phá toàn cục.

Không gian tìm kiếm cho các siêu tham số của SVM được định nghĩa trên thang logarit để đảm bảo tìm kiếm hiệu quả hơn trên các vùng có độ lớn khác nhau. Tham số điều chuẩn C được tìm kiếm trong khoảng $\log_{10}(C) \in [-2, 4]$, tương ứng với $C \in [10^{-2}, 10^4]$, cho phép thuật toán khám phá từ các giá trị rất nhỏ (điều chuẩn mạnh) đến các giá trị rất lớn

(điều chuẩn yếu). Tương tự, tham số gamma của kernel RBF được tìm kiếm trong khoảng $\log_{10}(\gamma) \in [-4, 1]$, tương ứng với $\gamma \in [10^{-4}, 10^{1}]$, bao phủ từ các giá trị nhỏ (ảnh hưởng rộng của mỗi điểm dữ liệu) đến các giá trị lớn (ảnh hưởng cục bộ). Việc tìm kiếm trên thang logarit giúp thuật toán khám phá đều hơn các vùng có độ lớn khác nhau của không gian tham số, tránh tình trạng tập trung quá nhiều vào các vùng có giá trị lớn và bỏ qua các vùng có giá trị nhỏ nhưng có thể chứa nghiệm tối ưu [10].

3 Thực nghiệm

3.1 Tập dữ liệu

Nghiên cứu này sử dụng hai tập dữ liệu để đánh giá hiệu suất của phương pháp đề xuất trong hai bối cảnh khác nhau: huấn luyện và tối ưu siêu tham số trên một tập dữ liệu, sau đó đánh giá khả năng tổng quát hóa trên một tập dữ liệu khác từ nguồn khác biệt.

3.1.1 Tập dữ liệu Vietnamese Online News (Kaggle)

Tập dữ liệu chính được sử dụng trong nghiên cứu là Vietnamese Online News Dataset, một tập ngữ liệu các bài báo tin tức tiếng Việt được công khai trên Kaggle. Tập dữ liệu này ban đầu chứa 184.539 bài báo tin tức trực tuyến được thu thập từ 25 trang tin tức phổ biến nhất tại Việt Nam vào tháng 7 năm 2022, với các bài báo được gán nhãn thủ công vào 16 chủ đề khác nhau. Tuy nhiên, để đảm bảo chất lượng dữ liệu và cân bằng giữa các lớp, nghiên cứu này thực hiện quy trình tiền xử lý và lọc dữ liệu có hệ thống.

Đầu tiên, các bài báo trùng lặp được loại bỏ để tránh hiện tượng data leakage giữa các tập train, validation và test, đồng thời các bản ghi chứa dữ liệu trống ở các cột quan trọng cũng được loại bỏ. Tiếp theo, các chủ đề có số lượng bài báo quá ít (dưới 3.000 bài) được loại bỏ để đảm bảo mỗi lớp có đủ dữ liệu cho việc huấn luyện và đánh giá mô hình một cách đáng tin cậy. Cuối cùng, kỹ thuật undersampling được áp dụng để cân bằng số lượng mẫu giữa các lớp, tránh hiện tượng mô hình thiên vị về các lớp có nhiều dữ liệu hơn. Sau quá trình tiền xử lý, tập dữ liệu cuối cùng gồm **37.860 bài báo** được phân loại đều vào **10 chủ đề**, với mỗi chủ đề có 3.786 bài báo. Các chủ đề bao gồm: *Chính trị* (Politics), *Công nghệ* (Technology), *Giáo dục* (Education), *Kinh tế* (Economics), *Pháp luật* (Law), *Sức khỏe đời sống* (Health and Lifestyle), *Thế giới* (World News), *Thể thao* (Sports), *Văn hóa giải trí* (Culture and Entertainment), và *Xe* (Automobiles). Tập dữ liệu này được sử dụng cho cả quá trình huấn luyện mô hình và tìm kiếm siêu tham số tối ưu thông qua thuật toán Cuckoo Search.

3.1.2 Tập dữ liệu Thanh Niên (Out-of-domain test set)

Để đánh giá khả năng tổng quát hóa của mô hình trên dữ liệu từ nguồn khác, một tập dữ liệu thứ hai được thu thập từ báo điện tử Thanh Niên (Thanh Nien Online), một trong những trang tin tức uy tín và có lượng độc giả lớn tại Việt Nam. Tập dữ liệu này đóng vai trò như một tập kiểm tra ngoài miền (out-of-domain test set), giúp đánh giá khả năng của mô hình khi áp dụng vào thực tế với dữ liệu từ nguồn khác biệt về phong cách viết, cấu trúc bài báo và từ vựng sử dụng.

Quy trình thu thập bắt đầu với việc lấy các bài báo từ các chuyên mục khác nhau của báo Thanh Niên, đảm bảo bao phủ đầy đủ 10 chủ đề tương ứng với tập Kaggle. Dữ liệu sau đó được làm sạch một cách cẩn thận thông qua nhiều bước xử lý: các bài báo trùng lặp được loại

bỏ để đảm bảo tính độc lập của dữ liệu; Unicode được chuẩn hóa để đảm bảo tính nhất quán trong mã hóa ký tự và chữ viết hoa được chuẩn hóa; các ký tự đặc biệt và thẻ HTML còn sót lại được loại bỏ hoàn toàn. Các bài báo sau đó được ánh xạ vào hệ thống phân loại 10 chủ đề tương tự với tập Kaggle, đảm bảo tính nhất quán trong việc đánh giá hiệu suất mô hình. Quan trọng nhất, tập dữ liệu Thanh Niên được giữ lại hoàn toàn riêng biệt và không được sử dụng trong bất kỳ giai đoạn nào của quá trình huấn luyện hay tối ưu siêu tham số. Điều này đảm bảo rằng kết quả đánh giá phản ánh chính xác khả năng tổng quát hóa của mô hình trong điều kiện thực tế, khi mô hình phải xử lý dữ liệu từ nguồn mà nó chưa từng tiếp xúc trong quá trình học.

3.2 Thiết lập thực nghiệm

3.2.1 Phân chia dữ liệu

Tập dữ liệu Vietnamese Online News gồm 37.860 bài báo được phân chia theo chiến lược 5-fold cross-validation để đảm bảo đánh giá toàn diện và khách quan hiệu suất của mô hình. Toàn bộ tập dữ liệu được chia ngẫu nhiên thành 5 phần (folds) có kích thước bằng nhau, trong đó mỗi phần chứa khoảng 7.572 bài báo. Trong mỗi vòng lặp, 4 phần tương đương 80% dữ liệu (30.288 bài báo) được sử dụng cho huấn luyện, trong khi 1 phần còn lại tương đương 20% dữ liệu (7.572 bài báo) được sử dụng cho kiểm tra. Quá trình này được lặp lại 5 lần, mỗi lần sử dụng một phần khác nhau làm tập kiểm tra, đảm bảo rằng mỗi mẫu dữ liệu đều được sử dụng đúng một lần cho kiểm tra và bốn lần cho huấn luyện. Kết quả cuối cùng được tính là trung bình của 5 vòng lặp, giúp giảm thiểu ảnh hưởng của việc phân chia dữ liệu ngẫu nhiên và cung cấp ước lượng đáng tin cậy về khả năng tổng quát hóa của mô hình.

3.2.2 Trích xuất đặc trưng với PhoBERT

Mô hình PhoBERT-base được sử dụng để chuyển đổi văn bản thô thành các vector đặc trưng số học có khả năng biểu diễn ngữ nghĩa của văn bản tiếng Việt. PhoBERT-base có kiến trúc gồm 12 lớp Transformer với 768 hidden units và 12 attention heads, được huấn luyện trước trên kho ngữ liệu lớn tiếng Việt. Văn bản đầu vào được tiền xử lý theo quy trình 5 bước đã mô tả trong Mục 2.2, sau đó được token hóa bằng tokenizer của PhoBERT với độ dài tối đa 256 tokens để đảm bảo cân bằng giữa khả năng nắm bắt ngữ cảnh và hiệu quả tính toán. Vector biểu diễn của token [CLS] tại lớp cuối cùng, có 768 chiều, được trích xuất và sử dụng làm đặc trưng đại diện cho toàn bộ văn bản. Các trọng số của mô hình PhoBERT được giữ cố định (frozen) và không được fine-tune trong quá trình huấn luyện, điều này giúp giảm chi phí tính toán và tránh overfitting trên tập dữ liệu có kích thước hạn chế.

3.2.3 Mô hình baseline và mô hình đề xuất

Để đánh giá hiệu quả của việc tối ưu siêu tham số bằng Cuckoo Search, hai mô hình SVM được xây dựng và so sánh một cách có hệ thống. Mô hình baseline (SVM-Baseline) sử dụng SVM với kernel tuyến tính (Linear kernel) và các siêu tham số mặc định từ thư viện scikit-learn, cụ thể là tham số điều chuẩn $C = 1.0$ và không có tham số γ do sử dụng kernel tuyến tính. Mô hình này đại diện cho cách tiếp cận thông thường khi áp dụng SVM mà không có bước tối ưu siêu tham số, giúp làm nổi bật lợi ích của việc tối ưu hóa.

Mô hình đề xuất (CS-SVM) sử dụng SVM với kernel RBF, được lựa chọn dựa trên khuyến nghị cho các bài toán phân loại phức tạp [10], với các siêu tham số được tối ưu bởi thuật toán

Cuckoo Search. Tham số điều chuẩn C được tìm kiếm trong khoảng rộng từ 10^{-2} đến 10^4 để khám phá từ điều chuẩn mạnh đến điều chuẩn yếu, trong khi tham số kernel γ được tìm kiếm trong khoảng từ 10^{-4} đến 10^1 để cân bằng giữa ảnh hưởng cục bộ và ảnh hưởng rộng của các điểm dữ liệu. Thuật toán Cuckoo Search được cấu hình với kích thước quần thể $n = 20$ để đảm bảo đa dạng không gian tìm kiếm, số vòng lặp $t_{\max} = 5$ để cân bằng giữa thời gian thực thi và khả năng hội tụ, và tỷ lệ phát hiện $P_a = 0.25$ để duy trì sự cân bằng giữa khám phá và khai thác. Cả hai mô hình đều sử dụng chiến lược đa lớp One-vs-One (OvO) để xử lý bài toán phân loại 10 lớp, trong đó mỗi cặp lớp được phân biệt bởi một bộ phân loại nhị phân riêng biệt.

3.2.4 Quy trình tối ưu siêu tham số

Đối với mô hình CS-SVM, quy trình tối ưu siêu tham số được thực hiện một cách cẩn thận trong mỗi fold của cross-validation để tránh data leakage và đảm bảo tính khách quan của đánh giá. Đầu tiên, tập huấn luyện chiếm 80% dữ liệu được chia tiếp thành hai phần: 80% cho train nội bộ và 20% cho validation nội bộ, tạo ra môi trường tối ưu hóa độc lập với tập kiểm tra chính. Quần thể của thuật toán Cuckoo Search được khởi tạo với $n = 20$ bộ siêu tham số được chọn ngẫu nhiên từ không gian tìm kiếm trên thang logarit.

Trong mỗi vòng lặp của thuật toán CS, mỗi bộ siêu tham số trong quần thể được đánh giá thông qua việc huấn luyện mô hình SVM trên tập train nội bộ và đo hiệu suất trên tập validation nội bộ, với fitness function được định nghĩa là negative accuracy để chuyển bài toán tối đa hóa accuracy thành bài toán tối thiểu hóa phù hợp với bản chất của thuật toán tối ưu. Các bước đặc trưng của thuật toán CS, bao gồm Lévy flights để tạo ra các giải pháp mới thông qua bước nhảy ngẫu nhiên có phân phối nặng đuôi, và cơ chế abandonment để loại bỏ các giải pháp kém và thay thế bằng các giải pháp mới, được thực hiện để cập nhật quần thể theo hướng cải thiện fitness. Quá trình này lặp lại cho đến khi đạt $t_{\max} = 5$ vòng lặp, sau đó bộ siêu tham số có fitness tốt nhất được chọn làm cấu hình tối ưu.

Cuối cùng, mô hình SVM được huấn luyện lại với bộ siêu tham số tối ưu này trên toàn bộ tập huấn luyện (80% dữ liệu ban đầu), không chỉ trên tập train nội bộ, để tận dụng tối đa lượng dữ liệu có sẵn và cải thiện khả năng học của mô hình. Mô hình cuối cùng này sau đó được đánh giá trên tập kiểm tra (20% dữ liệu ban đầu) để đo lường hiệu suất thực tế. Quy trình này được lặp lại cho cả 5 folds của cross-validation, đảm bảo rằng việc tối ưu siêu tham số được thực hiện độc lập trong mỗi fold và kết quả cuối cùng phản ánh chính xác khả năng tổng quát hóa của phương pháp đề xuất.

3.3 Chỉ số đánh giá

Để đánh giá toàn diện hiệu suất của các mô hình, nghiên cứu sử dụng nhiều chỉ số đánh giá khác nhau, bao gồm các chỉ số tổng thể và các chỉ số chi tiết theo từng lớp.

3.3.1 Chỉ số đánh giá tổng thể

Accuracy (Độ chính xác):

$$\text{Accuracy} = \frac{\text{Số mẫu được phân loại đúng}}{\text{Tổng số mẫu}} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{22}$$

trong đó TP (True Positive), TN (True Negative), FP (False Positive), và FN (False Negative) là các giá trị từ ma trận nhầm lẫn (confusion matrix).

Macro-averaged Precision (Precision macro):

$$\text{Precision}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K}\frac{TP_k}{TP_k + FP_k}, \tag{23}$$

trong đó K là số lớp (trong nghiên cứu này $K = 10$), và TP_k, FP_k là các giá trị True Positive và False Positive cho lớp k.

Macro-averaged Recall (Recall macro):

$$\text{Recall}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K}\frac{TP_k}{TP_k + FN_k}. \tag{24}$$

Macro-averaged F1-score (F1 macro):

$$\text{F1}_{\text{macro}} = \frac{1}{K}\sum_{k=1}^{K}\frac{2 \times \text{Precision}_k \times \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}. \tag{25}$$

Macro-averaging được sử dụng thay vì micro-averaging hoặc weighted-averaging để đảm bảo rằng tất cả các lớp được đánh giá công bằng, không bị ảnh hưởng bởi sự mất cân bằng dữ liệu (mặc dù trong nghiên cứu này dữ liệu đã được cân bằng).

3.3.2 Ma trận nhầm lẫn (Confusion Matrix)

Ma trận nhầm lẫn $\mathbf{C} \in \mathbb{R}^{K \times K}$ được sử dụng để phân tích chi tiết hiệu suất của mô hình trên từng lớp, cung cấp cái nhìn sâu sắc về khả năng phân biệt các chủ đề khác nhau của mô hình. Trong ma trận này, phần tử C_{ij} biểu diễn số lượng mẫu thực tế thuộc lớp i nhưng được mô hình dự đoán là lớp j. Các phần tử trên đường chéo chính C_{ii} đại diện cho số lượng mẫu được phân loại đúng của lớp i, do đó giá trị cao trên đường chéo cho thấy mô hình có hiệu suất tốt trên lớp tương ứng. Ngược lại, các phần tử ngoài đường chéo C_{ij} với $i \neq j$ cho biết số lượng mẫu bị phân loại sai, cụ thể là mô hình thường nhầm lẫn giữa lớp i và lớp j. Thông qua việc phân tích các giá trị ngoài đường chéo, ta có thể xác định các cặp lớp dễ bị nhầm lẫn, thường là do sự tương đồng về nội dung hoặc từ vựng giữa các chủ đề. Ma trận nhầm lẫn không chỉ cung cấp thông tin tổng quan về độ chính xác mà còn giúp hiểu rõ điểm mạnh và điểm yếu của mô hình trên từng chủ đề cụ thể, từ đó đưa ra các hướng cải thiện phù hợp cho các lớp có hiệu suất thấp.

4 Kết quả

4.1 Đánh giá mô hình cơ sở trên tập dữ liệu Kaggle

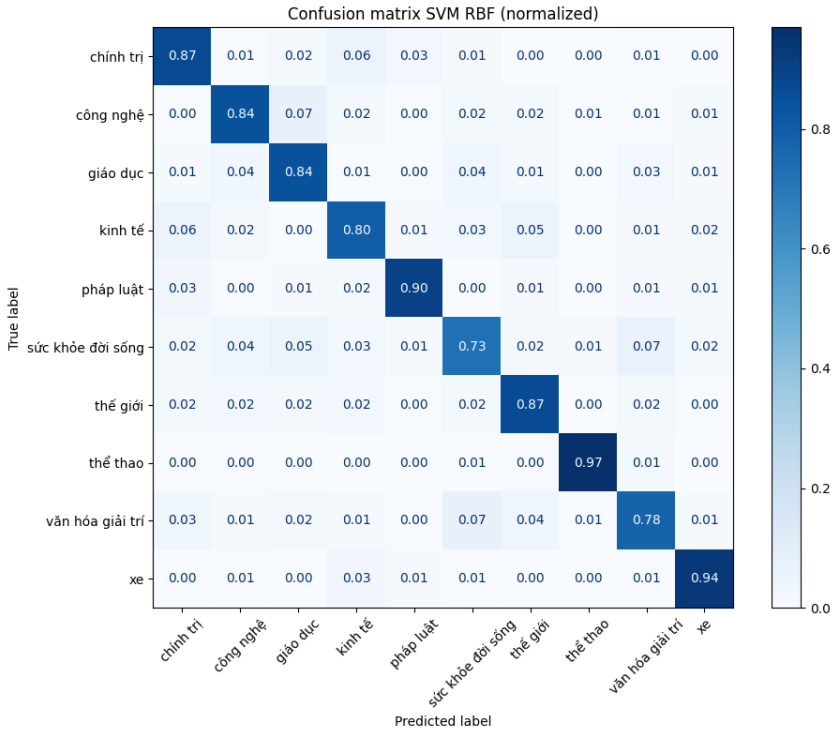
Chúng tôi bắt đầu bằng việc thiết lập một mô hình cơ sở sử dụng SVM chuẩn với nhân RBF trên tập dữ liệu tin tức Kaggle, bao gồm mười chủ đề tiếng Việt. Mô hình đạt độ chính xác tổng thể là 0.85 và điểm F1 macro là 0.85. Hiệu suất ở mức độ từng lớp tương đối cân bằng,

với điểm F1 dao động từ 0.73 đến 0.97. Kết quả tốt nhất đạt được ở các lớp *thể thao / sports* (F1 = 0.97), *xe / automobiles* (F1 = 0.93), và *pháp luật / law* (F1 = 0.91). Hiệu suất thấp hơn xuất hiện ở các lớp *sức khỏe đời sống / health and lifestyle* (F1 = 0.75) và *văn hóa giải trí / culture and entertainment* (F1 = 0.80), do các lớp này có sự biến đổi về phong cách và chủ đề lớn hơn.

Bảng 1 trình bày báo cáo phân loại chi tiết. Ma trận nhầm lẫn chuẩn hóa trong Hình 1 cho thấy hầu hết các lỗi xảy ra giữa các danh mục có liên quan về ngữ nghĩa. Các ví dụ điển hình bao gồm *kinh tế / economics* so với *pháp luật / law* và *văn hóa giải trí / culture and entertainment* so với *sức khỏe đời sống / health and lifestyle*. Mặc dù có những chồng chéo này, cấu trúc đường chéo mạnh cho thấy mô hình SVM cơ sở đã nắm bắt được ranh giới chủ đề rõ ràng trong không gian embedding PhoBERT và do đó cung cấp một điểm tham chiếu đáng tin cậy trước khi áp dụng tối ưu hóa siêu tham số.

Bảng 1 Báo cáo phân loại cho mô hình SVM cơ sở với nhân RBF trên tập dữ liệu Kaggle.

Lớp	**Precision**	**Recall**	**F1-score**
chính trị	0.82	0.87	0.85
công nghệ	0.84	0.84	0.84
giáo dục	0.81	0.84	0.83
kinh tế	0.80	0.80	0.80
pháp luật	0.92	0.90	0.91
sức khỏe đời sống	0.78	0.73	0.75
thế giới	0.84	0.87	0.86
thể thao	0.98	0.97	0.97
văn hóa giải trí	0.83	0.78	0.80
xe	0.91	0.94	0.93
accuracy	—	—	0.85
macro avg	0.85	0.85	0.85
weighted avg	0.85	0.85	0.85



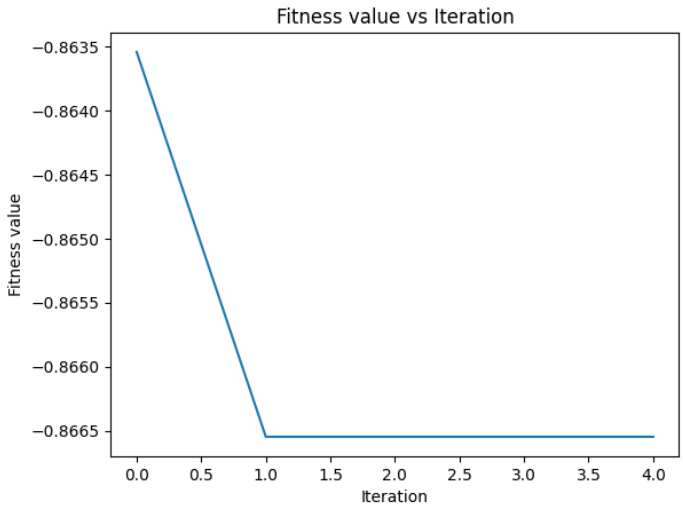
Hình 1 Ma trận nhầm lẫn chuẩn hóa cho mô hình SVM cơ sở với nhân RBF trên tập dữ liệu Kaggle.

4.2 Mô hình SVM được tối ưu hóa bằng CS trên tập dữ liệu Kaggle

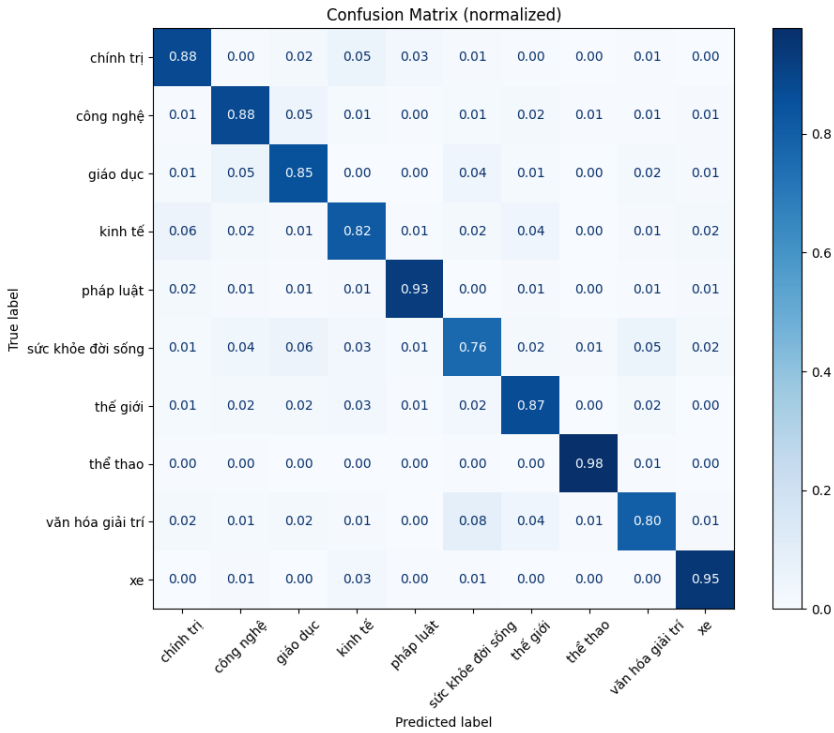
Chúng tôi áp dụng thuật toán Cuckoo Search để tối ưu hóa các siêu tham số của bộ phân loại RBF-SVM. Quá trình tìm kiếm khám phá 157 tổ hợp tham số và hội tụ nhanh chóng, như thể hiện trong đường cong fitness ở Hình 2, ổn định sau lần lặp đầu tiên. Cấu hình tốt nhất đạt được tại $\log_{10}(C) = 0.4080$ và $\log_{10}(\gamma) = 0.6679$, tương ứng với $C = 2.55888$ và $\gamma = 4.65463$.

Với các tham số được tối ưu hóa này, mô hình đạt độ chính xác 0.87 và điểm F1 macro là 0.87, cải thiện so với mô hình cơ sở lên hai điểm phần trăm. Kết quả ở mức độ từng lớp trong Bảng 2 cho thấy sự cải thiện nhất quán trên hầu hết các danh mục. Các cải thiện đáng chú ý nhất xảy ra ở *sức khỏe đời sống / health and lifestyle* (F1 = 0.75 $\rightarrow$ 0.77) và *văn hóa giải trí / culture and entertainment* (F1 = 0.80 $\rightarrow$ 0.83), hai danh mục trước đây có tỷ lệ phân loại sai cao hơn.

Ma trận nhầm lẫn trong Hình 3 hiển thị sự thống trị đường chéo rõ ràng hơn so với mô hình cơ sở, đặc biệt đối với các danh mục có tần suất trung bình như *chính trị* và *giáo dục*. Các phân loại sai chủ yếu vẫn nằm giữa các chủ đề gần nhau về ngữ nghĩa, nhưng nhìn chung, Cuckoo Search mang lại ranh giới quyết định ổn định hơn trong không gian embedding PhoBERT.



Hình 2 Giá trị fitness qua các lần lặp CS.



Hình 3 Ma trận nhầm lẫn chuẩn hóa cho mô hình SVM được tối ưu hóa bằng CS trên tập dữ liệu Kaggle.

Bảng 2 Báo cáo phân loại cho mô hình SVM được tối ưu hóa bằng CS trên tập dữ liệu Kaggle.

Lớp	Precision	Recall	F1-score
chính trị	0.86	0.88	0.87
công nghệ	0.84	0.88	0.86
giáo dục	0.83	0.85	0.84
kinh tế	0.83	0.82	0.83
pháp luật	0.93	0.93	0.93
sức khỏe đời sống	0.79	0.76	0.77
thế giới	0.86	0.87	0.86
thể thao	0.98	0.98	0.98
văn hóa giải trí	0.86	0.80	0.83
xe	0.93	0.95	0.94
accuracy	—	—	0.87
macro avg	0.87	0.87	0.87
weighted avg	0.87	0.87	0.87

4.3 Đánh giá thực tế trên tập dữ liệu Thanh Niên

Để đánh giá khả năng tổng quát hóa của mô hình được tối ưu hóa, chúng tôi đánh giá SVM được điều chỉnh bằng CS trên một kho ngữ liệu thực tế gồm các bài báo thu thập từ báo điện tử Thanh Niên. Tập dữ liệu này khác với dữ liệu chuẩn Kaggle về phong cách viết, phân bố chủ đề và mức độ nhiễu, do đó đại diện cho một bài kiểm tra có ý nghĩa về tính bền vững dưới sự thay đổi miền. Mô hình cơ sở không được đánh giá ở đây, vì mục đích của thí nghiệm này là kiểm tra cấu hình sẵn sàng triển khai của hệ thống.

Trên tập dữ liệu ngoài miền này, mô hình được tối ưu hóa đạt độ chính xác tổng thể là 0.78 và điểm F1 macro là 0.79. Mặc dù thấp hơn kết quả trong miền, hiệu suất vẫn ổn định trên hầu hết các danh mục, cho thấy rằng việc tối ưu hóa Cuckoo Search chuyển giao khá tốt sang dữ liệu tin tức phân bố tự nhiên. Điểm số cao nhất được quan sát thấy ở *thể thao / sports* (F1 = 0.95), *xe / automobiles* (F1 = 0.91), và *pháp luật / law* (F1 = 0.92). Ngược lại, sự suy giảm đáng kể xuất hiện ở *sức khỏe đời sống / health and lifestyle* (F1 = 0.53) và *thế giới / world news* (F1 = 0.70), các danh mục có sự biến đổi chủ đề rộng hơn và cấu trúc ngôn ngữ ít nhất quán hơn.

Ma trận nhầm lẫn trong Hình 4 cung cấp thêm cái nhìn sâu sắc về các mẫu này. Các phân loại sai tăng lên đặc biệt giữa *công nghệ / technology* và *thế giới / world news*, cũng như giữa *sức khỏe đời sống / health and lifestyle* và một số danh mục có tần suất trung bình. Mặc dù có những thách thức này, xu hướng đường chéo vẫn chiếm ưu thế đối với hầu hết các lớp, cho thấy ranh giới quyết định được tối ưu hóa vẫn duy trì cấu trúc phân biệt ngay cả khi có sự không khớp về phân bố.

Bảng 3 Báo cáo phân loại cho mô hình SVM được tối ưu hóa bằng CS trên tập dữ liệu thực tế Thanh Niên.

Lớp	Precision	Recall	F1-score
chính trị	0.79	0.88	0.83
công nghệ	0.87	0.62	0.72
giáo dục	0.79	0.84	0.81
kinh tế	0.71	0.69	0.70
pháp luật	0.90	0.95	0.92
sức khỏe đời sống	0.45	0.65	0.53
thế giới	0.99	0.54	0.70
thể thao	0.99	0.92	0.95
văn hóa giải trí	0.75	0.80	0.78
xe	0.86	0.96	0.91
accuracy	—	—	0.78
macro avg	0.81	0.78	0.79
weighted avg	0.81	0.78	0.79

Hình 4 Ma trận nhầm lẫn chuẩn hóa của mô hình SVM được tối ưu hóa bằng CS trên tập dữ liệu thực tế Thanh Niên.

5 Thảo luận

Bảng 4 tóm tắt hiệu suất trên tất cả các thiết lập đánh giá. Trên dữ liệu chuẩn Kaggle, mô hình SVM được tối ưu hóa bằng CS cung cấp sự cải thiện rõ ràng so với mô hình cơ sở, tăng

cả độ chính xác và điểm F1 macro lên khoảng hai điểm phần trăm. Mặc dù mức tăng về số liệu là khiêm tốn, sự cải thiện này nhất quán trên gần như tất cả các danh mục chủ đề, cho thấy rằng các siêu tham số được chọn bởi Cuckoo Search nâng cao khả năng phân tách của các embedding PhoBERT trong không gian đặc trưng RBF. Các ma trận nhầm lẫn cũng chỉ ra rằng mô hình được tối ưu hóa làm giảm sự rò rỉ giữa các danh mục ở một số chủ đề có tần suất trung bình như *chính trị*, *giáo dục*, và *văn hóa giải trí*.

Khi chuyển giao sang tập dữ liệu thực tế Thanh Niên, mô hình được tối ưu hóa đạt độ chính xác 0.78 và điểm F1 macro là 0.79. Mặc dù thấp hơn kết quả trong miền, hiệu suất vẫn ổn định trên hầu hết các danh mục, cho thấy rằng quá trình tối ưu hóa tổng quát hóa tốt vượt ra ngoài các điều kiện kiểm soát của tập dữ liệu chuẩn. Sự giảm độ chính xác chủ yếu do sự trôi dạt chủ đề và sự biến đổi ngôn ngữ rộng hơn, đặc biệt ở *sức khỏe đời sống* và *thế giới*, nơi phong cách bài viết và phạm vi nội dung khác biệt đáng kể so với kho ngữ liệu Kaggle.

Nhìn chung, những phát hiện này cho thấy việc điều chỉnh metaheuristic mang lại lợi ích có ý nghĩa cho các bộ phân loại SVM được xây dựng trên các embedding dựa trên transformer. Cuckoo Search cải thiện khả năng hiệu chỉnh mô hình mà không làm tăng độ phức tạp của mô hình và tạo ra ranh giới quyết định bền vững hơn, chuyển giao khá tốt sang các tập dữ liệu bên ngoài. Kết quả cũng nhấn mạnh tầm quan trọng của việc kết hợp các dữ liệu chuẩn được tuyển chọn với các đánh giá thực tế khi đánh giá các hệ thống phân loại văn bản tiếng Việt, vì hiệu suất trên dữ liệu chuẩn có thể không dự đoán đầy đủ hành vi dưới các sự thay đổi phân bố xảy ra tự nhiên.

Bảng 4 So sánh giữa các mô hình SVM cơ sở và được tối ưu hóa bằng CS.

Mô hình	Accuracy	Macro F1-score
SVM cơ sở (Kaggle)	0.85	0.85
SVM tối ưu hóa CS (Kaggle)	0.87	0.87
SVM tối ưu hóa CS (Thanh Niên)	0.78	0.79

6 Kết luận

Bài báo này trình bày một khung làm việc được điều khiển bởi Cuckoo Search để tối ưu hóa siêu tham số cho các bộ phân loại SVM được xây dựng trên embedding PhoBERT cho bài toán phân loại tin tức tiếng Việt. Bằng cách kết hợp một mô hình transformer đơn ngữ mạnh với một metaheuristic dựa trên quần thể, phương pháp được đề xuất nhắm trực tiếp vào một trong những điểm nghẽn chính trong SVM nhân: độ nhạy cảm với các siêu tham số như hệ số điều chuẩn C và hệ số RBF γ.

Trên tập dữ liệu chuẩn Kaggle, mô hình SVM được tối ưu hóa bằng CS luôn vượt trội hơn mô hình SVM cơ sở được điều chỉnh cẩn thận, cải thiện cả độ chính xác và điểm F1 macro từ 0.85 lên 0.87. Sự cải thiện được quan sát thấy trên hầu hết các danh mục chủ đề và được xác nhận bởi phân tích ma trận nhầm lẫn, cho thấy sự nhầm lẫn giữa các chủ đề giảm, đặc biệt ở các lớp có tần suất trung bình. Khi được đánh giá trên kho ngữ liệu thực tế của báo điện tử Thanh Niên, mô hình được tối ưu hóa đạt độ chính xác 0.78 và điểm F1 macro là 0.79, cho

thấy rằng các siêu tham số được chọn bởi Cuckoo Search chuyển giao khá tốt dưới sự thay đổi phân bố và sự biến đổi ngôn ngữ tăng lên.

Những kết quả này cho thấy tối ưu hóa metaheuristic là một phương pháp bổ sung thực tế và hiệu quả cho các mô hình ngôn ngữ tiếng Việt hiện đại, đặc biệt trong các bối cảnh ít tài nguyên nơi việc tìm kiếm toàn diện là không khả thi và dữ liệu được gán nhãn bị hạn chế. Đồng thời, sự sụt giảm hiệu suất trên tập dữ liệu thực tế làm nổi bật những thách thức còn lại liên quan đến thích ứng miền, sự trôi dạt chủ đề và nhãn nhiễu.

Các nghiên cứu trong tương lai sẽ mở rộng hướng nghiên cứu này theo nhiều hướng. Thứ nhất, chúng tôi dự định nghiên cứu các metaheuristic thay thế và kết hợp, cũng như các công thức đa mục tiêu tối ưu hóa đồng thời độ chính xác và chi phí tính toán. Thứ hai, nghiên cứu hiện tại tập trung vào điều chỉnh một bộ phân loại SVM cố định; một hướng thú vị là tích hợp CS với việc tinh chỉnh nhẹ PhoBERT hoặc các kiến trúc dựa trên adapter. Cuối cùng, việc đánh giá khung làm việc được đề xuất trên các miền tiếng Việt bổ sung và kho ngữ liệu đa ngôn ngữ sẽ cung cấp một bức tranh toàn diện hơn về tính bền vững và khả năng áp dụng của nó trong các kịch bản NLP rộng hơn.

Tài liệu tham khảo

[1] Yang, L., Shami, A.: On hyperparameter optimization of machine learning algorithms: Theory and practice. Neurocomputing **415**, 295–316 (2020) https://doi.org/10.1016/j.neucom.2020.07.061

[2] Mantovani, R.G., Rossi, A.L.D., Vanschoren, J., Bischl, B., Carvalho, A.C.P.d.L.F.: Effectiveness of random search in svm hyper-parameter tuning. International Joint Conference on Neural Networks (IJCNN) (2015) https://doi.org/10.1109/IJCNN.2015.7280664

[3] Bergstra, J., Bengio, Y.: Random search for hyper-parameter optimization. Journal of Machine Learning Research **13**(1), 281–305 (2012)

[4] Huang, C.-L., Wang, C.-J.: A ga-based feature selection and parameters optimization for support vector machines. In: Expert Systems with Applications, vol. 31, pp. 231–240 (2006). https://doi.org/10.1016/j.eswa.2005.09.024

[5] Lin, S.-W., Ying, K.-C., Chen, S.-C., Lee, Z.-J.: Particle swarm optimization for parameter determination and feature selection of support vector machines. Expert Systems with Applications **35**(4), 1817–1824 (2008) https://doi.org/10.1016/j.eswa.2007.08.088

[6] Zhang, X., Chen, X., He, Z.: An aco-based algorithm for parameter optimization of support vector machines. Expert Systems with Applications **37**(9), 6618–6628 (2010) https://doi.org/10.1016/j.eswa.2010.03.067

[7] Rizkallah, L.W.: Optimizing svm hyperparameters for satellite imagery classification using metaheuristic and statistical techniques. International Journal of Data Science and Analytics (2025) https://doi.org/10.1007/s41060-025-00762-7

[8] Cortes, C., Vapnik, V.: Support-vector networks. Machine Learning **20**(3), 273–297

(1995)

[9] Gogoi, A., Baruah, N., Nath, L.J.: Assamese word sense disambiguation using cuckoo search algorithm. Procedia Computer Science **189**, 142–147 (2021) https://doi.org/10.1016/j.procs.2021.05.110

[10] Hsu, C.-W., Chang, C.-C., Lin, C.-J.: A practical guide to support vector classification. Technical Report, Department of Computer Science, National Taiwan University (2003)

[11] Yang, X.-S., Deb, S.: Cuckoo search via lévy flights. In: 2009 World Congress on Nature & Biologically Inspired Computing (NaBIC), pp. 210–214. IEEE, Coimbatore, India (2009)

[12] Yang, X.-S., Deb, S.: Engineering optimisation by cuckoo search. International Journal of Mathematical Modelling and Numerical Optimisation **1**(4), 330–343 (2010)

[13] Mantegna, R.N.: Fast, accurate algorithm for numerical simulation of lévy stable stochastic processes. Physical Review E **49**(5), 4677–4683 (1994) https://doi.org/10.1103/PhysRevE.49.4677

[14] Yang, X.-S., Deb, S.: Cuckoo search: Recent advances and applications. Neural Computing and Applications **24**(1), 169–174 (2014) https://doi.org/10.1007/s00521-013-1367-1

[15] Mareli, M., Twala, B.: An adaptive cuckoo search algorithm for optimisation. Applied Computing and Informatics **14**(2), 107–115 (2018) https://doi.org/10.1016/j.aci.2017.09.001

[16] Yang, Y., Tan, M., Dai, Y.: An improved cs-lssvm algorithm-based fault pattern recognition of ship power equipments. PLOS ONE **12**(2), 0171246 (2017) https://doi.org/10.1371/journal.pone.0171246

[17] Senthilkumar, S., Brindha, B.G.: An intelligent lung cancer diagnosis system using cuckoo search optimization and support vector machine classifier. Journal of Ambient Intelligence and Humanized Computing **9**(4), 1031–1042 (2018) https://doi.org/10.1007/s12652-017-0655-5

[18] Alshaher, J., Abdullah, R.: Pso-based support vector machine with cuckoo search technique for clinical disease diagnoses. Advances in Swarm Intelligence (2014) https://doi.org/10.1155/2014/548483

[19] Nguyen, D.Q., Nguyen, A.T.: Phobert: Pre-trained language models for vietnamese. In: Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1037–1042. Association for Computational Linguistics, Online (2020)